

Taller Práctico: Implementación y Análisis Asintótico de Algoritmos de Ordenamiento

1. Objetivo General

Desarrollar una aplicación interactiva que ordene un conjunto de datos masivo utilizando tres métodos clásicos de ordenamiento. El estudiante deberá medir el rendimiento empírico (tiempo de ejecución) y justificarlo mediante el análisis de complejidad teórica (Notación Asintótica: Big O, Big Θ , Big Ω).

2. Descripción del Problema

Se requiere construir un programa modular regido por un menú interactivo. El sistema debe ser capaz de cargar un gran volumen de datos desde una fuente externa (un archivo de texto plano .txt, un archivo .csv o una tabla de una base de datos) que contenga al menos **10,000 datos de personas desordenados devolver un archivo en formato Excel**.

Una vez cargados los datos, el programa debe ofrecer al usuario la posibilidad de elegir con qué algoritmo desea ordenarlos, calcular el tiempo exacto que tomó el proceso y emitir un reporte de la complejidad algorítmica teórica del método seleccionado.

3. Requerimientos Técnicos

Tu programa debe cumplir obligatoriamente con los siguientes puntos:

1. **Menú Principal:** Al iniciar, el programa debe mostrar las siguientes opciones:
 - [1] Cargar datos (Archivo o Base de Datos)
 - [2] Ordenar usando *Bubble Sort* (Burbuja)
 - [3] Ordenar usando *Insertion Sort* (Inserción)
 - [4] Ordenar usando *Selection Sort* (Selección)
 - [5] Salir
2. **Implementación Pura:** Está **estrictamente prohibido** utilizar funciones de ordenamiento
3. **Medición del Tiempo:** Al finalizar el ordenamiento, el programa debe imprimir en pantalla el tiempo exacto de ejecución en milisegundos o segundos (debes investigar cómo usar las librerías de tiempo de tu lenguaje de programación). *Nota: El tiempo de lectura del archivo NO debe incluirse en el tiempo de ordenamiento.*
4. **Reporte de Complejidad:** Junto con el tiempo de ejecución, el programa debe imprimir en pantalla la notación asintótica teórica del algoritmo elegido para sus tres casos:
Peor caso (Límite superior): Big O (n)
Caso promedio (Límite ajustado): Big $\Theta(n)$
Mejor caso (Límite inferior): Big $\Omega(n)$

4. Entregables

1. **Código Fuente:** Archivo con el código documentado.
2. **Archivo de Datos:** El archivo usado para las pruebas (mínimo 10,000 registros).
3. **Informe Breve (PDF):** Un documento donde incluyas una tabla comparando los tiempos de ejecución obtenidos al correr los tres algoritmos sobre el mismo conjunto de datos, y una conclusión de por qué uno fue más rápido que el otro basándote en la notación asintótica.